

Legged Robot Project

Till Beyer

Mattia Prandi

Andrea Tarabay

I. INTRODUCTION

Legged locomotion is challenging due to the high-dimensional states, hybrid dynamics, and the need for stability on diverse terrain. In quadruped robotics, two prominent control paradigms are biologically inspired Central Pattern Generators (CPGs) and Deep Reinforcement Learning (DRL). This report compares these approaches on a simulated quadruped.

CPGs are neural circuits capable of producing cyclic activity without rhythmic input [1]. In robotics, they are commonly implemented as coupled oscillators (e.g., Hopf oscillators) to generate coordinated limb motions [2]. CPGs provide stable gait generation and intuitive gait transitions. However, they can require extensive manual tuning and careful mapping from oscillator states to joint torques.

In contrast, DRL formulates locomotion as a Markov Decision Process (MDP), allowing policies to be learned directly from interaction with the environment [3]. Algorithms such as Proximal Policy Optimization (PPO) [4] and Soft Actor-Critic (SAC) [5] can produce highly robust behaviors over complex terrain. However, DRL methods typically suffer from high sample complexity, sensitivity to reward design, and the difficulty of traversing the *Sim-to-Real*-gap.

In this project, we implement and evaluate both approaches in a PyBullet simulation environment. First, we design a CPG-based controller using coupled Hopf oscillators to achieve stable locomotion in various gaits. Second, we design an MDP and train a neural network policy using DRL to track desired velocities and navigate uneven terrain tasks. We analyze the performance with respect to tracking accuracy, energy efficiency (Cost of Transport), and robustness to environmental irregularities.

II. METHODOLOGY

This section details the two control approaches implemented for quadruped locomotion: Central Pattern Generators (CPG) and Deep Reinforcement Learning (DRL).

A. Central Pattern Generators

We implement a biologically inspired controller based on coupled Hopf oscillators. This approach explicitly generates rhythmic limb trajectories which are then tracked by a low-level controller. To insure flexibility, we implement 4 distinct gaits: trot, walk, pace and bound, each differentiated solely by the phase offsets ϕ_{ij} .

1) *Hopf Oscillator Formulation:* The rhythmic signal for each leg i is generated by a Hopf oscillator, governed by the

following differential equations defining the amplitude r_i and phase θ_i :

$$\dot{r}_i = \alpha(\mu - r_i^2)r_i \quad (1)$$

$$\dot{\theta}_i = \omega_i + \sum_{j=0}^3 r_j w_{ij} \sin(\theta_j - \theta_i - \phi_{ij}) \quad (2)$$

where μ is the target amplitude, α is a convergence constant, and ω_i is the intrinsic frequency. The coupling term in (2) ensures synchronization between legs, where w_{ij} represents the coupling strength and ϕ_{ij} is the desired phase offset.

To achieve specific gaits, we define the phase offset matrix Φ containing ϕ_{ij} and switch the frequency ω_i between swing (ω_{sw}) and stance (ω_{st}) phases based on the duty cycle D .

2) *Trajectory Mapping:* The oscillator states (r_i, θ_i) are mapped to desired foot positions (x_d, z_d) in the leg frame. We utilize the mapping proposed in [2]:

$$x_d = -d_{\text{step}} r_i \cos(\theta_i) \quad (3)$$

$$z_d = \begin{cases} -h + g_c \sin(\theta_i) & \text{if } \sin(\theta_i) > 0 \text{ (swing)} \\ -h + g_p \sin(\theta_i) & \text{otherwise (stance)} \end{cases} \quad (4)$$

where h is the standing height, g_c is the ground clearance, and g_p is the ground penetration (used to stabilize stance).

3) *Low-Level Control:* The desired Cartesian foot positions p_d are converted to joint angles q_d via inverse kinematics. To track these, we compute the necessary torques τ both with a joint PD controller described by,

$$\tau = K_p(q_d - q) + K_d(\dot{q}_d - \dot{q}), \quad (5)$$

and with a cartesian PD controller following,

$$\tau = J(q)^\top [K_{p,\text{cart}}(p_d - p) + K_{d,\text{cart}}(\dot{p}_d - \dot{p})], \quad (6)$$

where K are appropriately tuned parameters and $J(q)^\top$ the Jacobian matrix for a given limb configuration q .

4) *Virtual Model Control:* To improve the stability and robustness of the CPG-based locomotion, we extend its functionality using Virtual Model Control (VMC). VMC introduces virtual mechanical components, such as springs and dampers, that interact with the robot's physical system. These components generate corrective forces to guide the robot toward stable, desirable behaviors. We follow the promising implementation presented in [6] and implement virtual springs in the following three ways:

- 1) *Attitude control:* To keep the robot's body parallel to the ground, we define springs attached to the four corners of a virtual plane through the trunk. When deviating from a

horizontal position, these springs introduce forces in the z -direction to counter the deviation. The corner points P of the robot-plane can be written as,

$$P = R \begin{bmatrix} 1 & 1 & -1 & -1 \\ 1 & -1 & 1 & -1 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad (7)$$

where R denotes the body rotation matrix. The resulting forces in z -direction for the four legs result from,

$$F_{z,att} = k_{att} \begin{bmatrix} 0 & 0 & 1 \end{bmatrix} P, \quad (8)$$

with the tuning gain k_{att} . This force is only applied when the respective leg is in contact with the ground.

- 2) Lateral angle of attack control: To further mitigate unwanted roll and pitch deviations, we implement a lateral angle of attack controller using virtual torsion springs at the hips. These springs cause corrective torques to align the legs vertically when viewed from the front. The torque applied to each leg is proportional to the difference between the body's roll rotation φ and the hip angle, defined as,

$$\tau_{lat} = k_{lat} (\varphi - q_{hip}). \quad (9)$$

In contrast to the attitude control, this method can be applied during both stance and swing phase.

- 3) Direction control: To prevent deviations from the goal direction, we lastly implement a directional control. We deviate from the approach taken in [6] in two ways. First, by defining lateral foot forces in y as proportional to the angular difference between the robot's current direction ψ and the desired yaw direction ψ_{des} . Second, we introduce an extra term proportional to the translational deviation, i.e. the difference between the robot's current y -position and the desired trajectory y_{des} (mostly = 0). This yields,

$$F_{y,dir} = k_{rot} \begin{bmatrix} -1 & -1 & 1 & 1 \end{bmatrix} (\psi_{des} - \psi) - k_{trans} \begin{bmatrix} 1 & 1 & 1 & 1 \end{bmatrix} (y_{des} - y), \quad (10)$$

with appropriate gains k_{rot} and k_{trans} .

To demonstrate the capabilities of CPG-based locomotion we conduct a series of tests. In the first, we simply demonstrate the functionality of the system. After that we tune sets of CPG-parameters for a variety of gaits. We do this both for a slow and stable locomotion as well as for the maximum possible forward velocity. Lastly we implement the above explained VMC and discuss the effects of extending the CPG approach with this and other kinds of feedback loops. These results are given in section III.

B. Deep Reinforcement Learning

To learn robust locomotion policies, we formulate the control problem as a Markov Decision Process (MDP) and solve it using the Proximal Policy Optimization (PPO) algorithm,

we decided to use PPO as it is the most used RL algorithm in literature. The architecture follows an Actor-Critic structure based on Multi-Layer Perceptrons (MLPs) with two hidden layers of 256 units and Tanh activation functions.

1) *Control Architectures*: We investigate two distinct parameterizations for the action space to compare the efficacy of learning end-effector positions versus modulating a biological rhythm.

a) *Cartesian PD Policy (cPD-RL)*: This architecture gives the agent direct control over the foot positions.

- Action Space (A_{cPD}): The action is a 12-dimensional vector $a_t \in \mathbb{R}^{12}$. The policy outputs residuals $(\Delta x, \Delta y, \Delta z)$ for each of the four feet. These residuals are added to the nominal foot positions in the leg frame and tracked by a Cartesian PD controller.
- Observation Space: The observation consists of proprioceptive measurements s_{prop} (joint positions, velocities, base orientation) and the previous action a_{t-1} .

b) *CPG-Modulated Policy (CPG-RL)*: This architecture combines the intrinsic stability of the Central Pattern Generator with the adaptability of RL.

- Action Space (A_{CPG}): The action is an 8-dimensional vector $a_t \in \mathbb{R}^8$. Instead of controlling joints or feet directly, the policy modulates the high-level CPG parameters: the intrinsic frequency ω and the target amplitude μ for each leg. This allows the agent to adjust the gait cycle duration and step height dynamically.
- Observation Space: To allow the policy to synchronize with the rhythmic signal, we augment the observation space with the internal CPG states:

$$S_{CPG} = \{s_{prop}, a_{t-1}, r, \theta, \dot{r}, \dot{\theta}\} \quad (11)$$

where r, θ and their derivatives represent the amplitude and phase of the oscillators.

2) *Forward velocity controller*: Our first task is to train a forward velocity controller using reinforcement learning. The goal is to track a commanded forward body speed v_x^* on flat terrain while maintaining stability, reducing lateral drift and yaw rotation, and limiting energy usage. We solve this task using Proximal Policy Optimization (PPO) and the Cartesian-PD policy.

a) *Reward design.*: In the forward velocity reward function, we remove the reward clipping $\max(\text{reward}, 0)$ so that the agent also receives negative feedback. This makes the undesired behaviors have a worse impact, which will potentially lead to more stable learning. We use a weighted sum of tracking and regularization terms:

$$r_t = r_{\text{track}} + r_{\text{alive}} - \lambda_{\text{back}} p_{\text{back}} - \lambda_{\text{lat}} p_{\text{lat}} - \lambda_{\omega} p_{\text{yaw}} - \lambda_{\text{rp}} p_{\text{rp}} - \lambda_E p_E - \lambda_{\Delta a} p_{\text{smooth}} - \lambda_{\text{stance}} p_{\text{stance}}. \quad (12)$$

Tracking is shaped using a bounded Gaussian kernel in the velocity error, following the formulation in [7].

$$r_{\text{track}} = \exp(-k(v_x - v_x^*)^2), \quad (13)$$

Reward term	Weight
r_{track} (velocity tracking)	+1.0
r_{alive} (alive bonus)	+0.05
p_{back} (backward)	-1.0
p_{lat} (lateral velocity)	-1.0
p_{yaw} (yaw-rate)	-0.5
p_{rp} (roll/pitch)	-0.5
p_E (energy)	-0.002
p_{smooth} (action smoothness)	-0.05
p_{stance} (stance regularization)	-1.0

TABLE I: Reward weights used for PPO velocity training.

which approaches 1 when $v_x \approx v_x^*$. Some behaviors are discouraged by adding penalties on backward motion $p_{\text{back}} = \max(0, -v_x)$, lateral drift $p_{\text{lat}} = v_y^2$, yaw rate $p_{\text{yaw}} = \omega_z^2$, and roll/pitch deviations $p_{\text{rp}} = \phi^2 + \theta^2$. Efficiency is encouraged with an energy proxy

$$p_E = \sum_j |\tau_j \dot{q}_j| \Delta t, \quad (14)$$

and command smoothness with an action-rate penalty

$$p_{\text{smooth}} = \frac{1}{N} \|a_t - a_{t-1}\|^2. \quad (15)$$

A small alive reward r_{alive} encourages continued locomotion, and an additional penalty is applied on termination (falling).

b) Stance regularization.: In our early trainings, we observed a non-ideal asymmetric stance configuration where the front legs adopted a different lateral width than the hind legs. To discourage this, we add a stance regularizer based on the lateral foot positions y_i (leg/body frame). In practice, p_{stance} penalizes deviation from nominal lateral placement, front-hind stance-width mismatch, and left-right asymmetry; this guides the policy toward more symmetric lateral foot placement without affecting the objective.

c) Training protocol.: We use a staged training procedure with checkpoint loading: (i) train at $v_x^* = 0.8$ m/s to obtain a stable baseline gait, (ii) fine-tune from this checkpoint at $v_x^* = 1.5$ m/s, then (iii) fine-tune again at 1.5 m/s with p_{stance} enabled to correct the front-hind stance-width imbalance.

d) Evaluation metrics.: We report the steady-state average v_x after a warmup period, cost of transport $\text{CoT} = E/(mgd)$, gait timing statistics (stride period, duty factor) extracted from foot-contact signals, and per-leg gait metrics.

3) Climbing stairs Task.: The objective of this task is to enable the robot to traverse a staircase while maintaining stability and a constant forward velocity. Given the complexity of the task, direct training on the target stairs often results in instability or sub-optimal local convergence. Therefore, we adopted a *Curriculum Learning* approach, structuring the training into phases of increasing difficulty:

- 1) Walking Policy: Initially, we ensured the robot was able to walk stably on flat terrain (using a policy for CPG).
- 2) Low Stairs: Subsequently, the robot was trained on stairs with low step heights (0.03 m). This intermediate step facilitates learning the necessary kinematics to lift the

legs without immediately facing the instability of high steps.

- 3) Nominal Stairs: Finally, the agent was trained on stairs with the nominal target dimensions (0.05 m).

To mitigate the *Sim-to-Real gap*, as proposed in literature [8], we used extensive domain randomization. This included adding noise to observations and randomizing physical properties such as mass, inertia, motor gains, and ground friction, along with variations in step dimensions (height and width). The Sim-to-Real gap will be explained in Discussion [IV-C3].

To achieve the desired behaviors across different tasks, we designed a reward function composed of a weighted sum of several terms. Every 10 steps (100 Hz), the total reward r_t is calculated as:

$$r_t = \sum_i \lambda_i \cdot \mathcal{R}_i(\mathbf{s}_t, \mathbf{a}_t) \quad (16)$$

where λ_i represents the weight of the i -th term. The individual reward components $\mathcal{R}_i$ are generalized as follows:

$$\begin{aligned} \mathcal{R}_{\text{vel}} &= \text{clip}(v_x, v_{\min}, v_{\max}) \\ \mathcal{R}_{\text{drift}} &= -|p_y| \\ \mathcal{R}_{\text{energy}} &= -\left(\sum_j |\tau_j \cdot \dot{q}_j|\right) \Delta t \\ \mathcal{R}_{\text{quat}} &= -\|\mathbf{q} - \mathbf{q}_I\| \\ \mathcal{R}_{\text{euler}}^{\text{pow}} &= -(\phi^2 + \psi^2)^{0.7} \\ \mathcal{R}_{\text{action}} &= -\sqrt{\frac{1}{N} \sum (\mathbf{a}_t - \mathbf{a}_{t-1})^2} \\ \mathcal{R}_{\text{vert}} &= \max\left(0, \tanh\left(\frac{p_z - h_{\text{target}}}{\sigma}\right)\right) \end{aligned}$$

Here, v_x is the base linear velocity, p_y, p_z are the base positions, and $\mathbf{q}$ is the base orientation quaternion ($\mathbf{q}_I = [0, 0, 0, 1]$). For Euler angles, ϕ denotes roll and ψ denotes yaw. $\boldsymbol{\tau}$ and $\dot{\mathbf{q}}$ represent motor torques and velocities.

To ensure balanced learning, the weights λ_i were tuned by analyzing the magnitude of individual reward components before and after training, as exemplified in the appendix in Fig. 13. This data-driven normalization prevents specific terms from dominating the total reward signal. The resulting configuration is detailed in Table II.

Term	Walk	Low Steps	High Steps
$\mathcal{R}_{\text{vel}}$	0.1	0.2	0.2
$\mathcal{R}_{\text{drift}}$	0.01	0.1	0.1
$\mathcal{R}_{\text{energy}}$	0.02	0.01	0.05
$\mathcal{R}_{\text{quat}}$	0.1	—	0.5
$\mathcal{R}_{\text{euler}}^{\text{pow}}$	—	0.5	—
$\mathcal{R}_{\text{action}}$	—	—	0.02
$\mathcal{R}_{\text{vert}}$	—	—	0.08

TABLE II: Reward weights (λ) configuration for different policies. Note that $v_{\max}$ is set to 1.0 m/s for the Walk policy and 0.5 m/s for the others.

III. RESULTS

A. CPG

The results of the three experiments demonstrating the capabilities of the CPG-locomotion are given below. We discuss and interpret our findings in Section IV.

1) *Functionality*: The CPG-states obtained from a noise-free, 3 second long *Trot* on even terrain are depicted in Fig. 1. As expected, the oscillator amplitudes r shown on the left swiftly rise to their maximum $\sqrt{\mu}$, while the derivative $\dot{r}$ returns to zero as soon as the amplitude has reached its maximum. The phases θ for the different legs are shown on the right. Immediately we can see the characteristic 180° phase shift between the FR-RL and FL-RR leg pairs. When looking closely, one can furthermore make out that this phase shift is not constant from the start but rather takes some time to develop. This follows from the phase definition in Eq. 1 and Eq. 2. For comparison, the walking gait's oscillator phases are depicted in Fig. 8 in the appendix. Here, after a short delay, the legs move with a constant phase shift of 90° to each other.

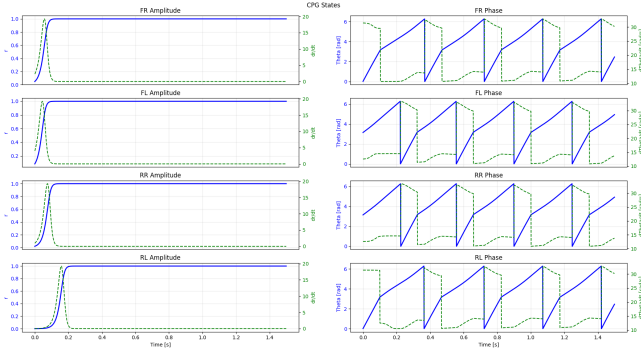


Fig. 1: Development of the CPG states for a trotting gait

We implemented both a joint- and cartesian PD controller to apply the foot positions calculated by the CPG to the robot. The tracking performance of each controller is shown in Fig. 2 for the foot position. For simplicity we have decided to conduct this test with the robot mounted to the rack, i.e. without ground contact. In the plot the theoretically desired configuration is shown in black while the actually implemented input is shown in blue. Clearly, even though the foot position is tracked precisely already without the cartesian PD, implementing it improves the tracking even more. We performed the same test for the joint angle tracking of the controllers. However, adding cartesian PD control did not make a significant difference. The plots are given in the appendix in Fig. 9a and Fig. 9b. For this demonstration we used the gains $K_p = 100$, $K_d = 2$ and $K_{p, \text{cart}} = 700$, $K_{d, \text{cart}} = 70$ (for all 3 dimensions respectively). These are the same values we found to be optimal in the velocity tuning, discussed in the next section.

2) Parameter Tuning for high and low velocity:

With hand-tuned parameters, our best results for the low- and high-speed tuning are listed in Table III. In each case the speed we used corresponds to the

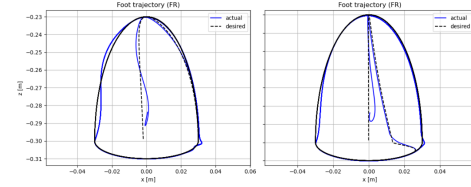


Fig. 2: Comparison of the Foot position for a trotting gait **with** (left) and **without** (right) Cartesian PD

speed after convergence of the gait sequence. Further, TROT_HIGH_1.8ms.mp4, WALK_HIGH_2.6ms.mp4 and BOUND_HIGH_1ms.mp4 show our best attempts for the trotting, walking and bounding gaits. For comparison TROT_LOW_0.1ms.mp4, WALK_LOW_0.13ms.mp4 BOUND_LOW_0.2ms.mp4 show the same gaits at slower speeds. From the high-speed results it is obvious that we are able to achieve the highest velocities with the walking gait although this comes at the price of a high CoT. For all gaits and speed we use similar Duty ratios around 0.7 and achieve the increase in speed mainly through the decreased Stride duration (and step length). A notable difference between the gaits is the CoT delta between low and high speed locomotion: For trotting and walking the CoT nearly doubles when going fast while for bounding it is cut in half. We discuss the reasons for these observations along with our choice of parameters in Section IV.

3) *Virtual Model Control*: To test and validate the VMC, we again apply the settings from Table V that lead to the maximum velocity. For this test we chose to focus on the walking gait, since it could achieve the highest velocities. We tune the VMC spring constants to $k_{att} = 300$, $k_{lat} = 200$, $k_{rot} = 100$, $k_{trans} = 100$ and observe and compare the roll/pitch-stability as well as the xy -trajectory for the system with and without VMC. The results of a 3-second VMC on/off-comparison are given in the appendix in Fig. 10a for the trajectory and in Fig. 10b for the orientation angles. From the trajectory it is clear that the implementation of VMC enables the robot to correct deviations from its desired heading direction. Furthermore, the evolution of the roll angle shows smaller ripples when using VMC implying improved stability around the roll axis. The pitch stability however does not change significantly. The video WALK_VMC_2.2ms.mp4 shows the VMC in action.

B. DRL - Velocity

The PPO training process evolves across three stages. In the first baseline stage, episode length and rewards converge to their maximum values after an initial learning period. During the second stage shown in Fig. 3, the agent maintains a stable gait while the reward is further refined. Finally, in the third stage, the agent sustains maximum rewards and the full 1000-step episode length from the very beginning of the training session. Plots displaying the training dynamics for Stages 1 and 3 are given in the appendix in Fig. 11.

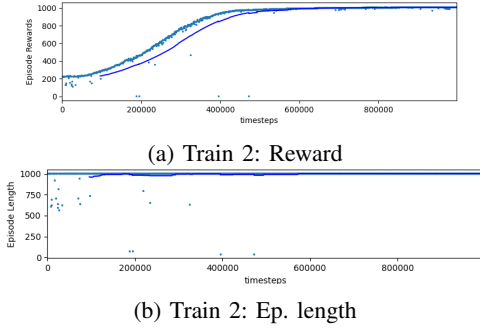


Fig. 3: PPO learning curves of running.

Gait	Speed	CoT	Duty ratio	Stride duration
Trot-high	1.8m/s	2.3	0.733	0.25s
Trot-low	0.1m/s	1.2	0.667	1.25s
Walk-high	2.6m/s	2.2	0.667	0.25s
Walk-low	0.13m/s	1.3	0.733	0.8s
Bound-high	1m/s	1.2	0.667	0.83s
Bound-low	0.2m/s	2.5	0.818	1.53s
RL-Velocity	1.5m/s	1.2	0.62	0.24s

TABLE III: Results for CPG-tuning and forward Velocity RL

The results achieved with our forward velocity tracking controller are shown in Table III and in Fig. 4. After a short transient at the start of the rollout, the policy accelerates to the desired velocity of $v_{\text{des}} = 1.5$ m/s and maintains it for the remainder of the episode with only small steady-state oscillations around the reference. At the same time, the lateral velocity v_y remains close to zero and shows only periodic gait-induced fluctuations. This induces a slight sideways drift. Overall, the learned controller achieves stable, near-constant forward locomotion at the target speed within the episode horizon.

The duty factor of 0.62 suggests a stance-dominant gait, which is typically associated with robustness at higher speeds. The per-leg duty differences mentioned in Table IV indicate the learned gait is not perfectly symmetric, which can correlate with residual lateral drift.

Lastly, the base trajectories are given in the appendix in Fig. 12a for a sideways view (XZ -plane) and in Fig. 12b for a top-down perspective (XY -plane). Both plots indicate stability though the sideways drift is clearly visible.

The video `RL_VEL_1.5ms.mp4` showing the final controller

C. DRL - Climbing Stairs

Following a series of training sessions, we obtained a policy capable of traversing diverse randomized staircases. The training dynamics are illustrated in Fig. 5. Although leveraging a pre-trained model facilitates general convergence, the dense scatter of points near zero in the episode length plot reveals significant initial instability. This indicates that the agent frequently encounters immediate failures (early termination) before mastering the survival strategy required for the irregular terrain. Over time, while the episode length

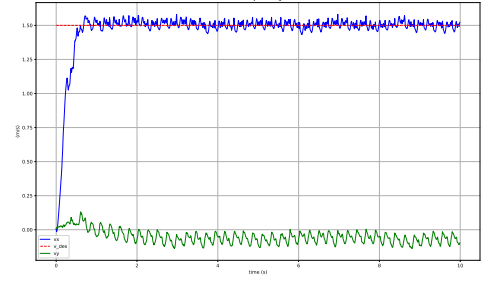


Fig. 4: Forward-velocity tracking

Leg	Duty ratio	T_{stride}
FR	0.487	0.231s
FL	0.607	0.236s
HR	0.696	0.235s
HL	0.711	0.229s

TABLE IV: Per-leg gait metrics for the learned velocity-tracking policy at $v_{\text{des}} = 1.5$ m/s.

saturates, the reward signal continues to rise, signifying the ongoing refinement of gait efficiency and stability.

The resulting feet trajectories are analyzed in Fig. 6. The feet trajectories display a smooth, cyclic profile characteristic of the underlying CPG modulation.

Cartesian base trajectories (Fig. 14) confirm locomotion stability. The XZ projection (Fig. 14a) shows a consistent linear ascent, while the XY view (Fig. 14b) demonstrates minimal lateral drift despite terrain perturbations.

Robustness was validated through extensive domain randomization, involving payload variations (0–6 kg, approximately 50% of the nominal mass) and stochastic step dimensions (height: 45–52 mm; length: 240–260 mm). Visual demonstrations of the robot’s performance are provided in the supplementary video `RL_STAIRS_0.64ms.mp4`. Additionally, the video `RL_CPG_0.73ms.mp4` shows the robot’s behavior after the initial flat-ground training stage

IV. DISCUSSION

A. CPG

1) *Control*: The evaluation of the joint PD and joint + cartesian PD control strategies revealed the following insights:

- Cartesian PD control noticeably improves the foot position tracking accuracy as seen in Fig. 2. This result was to be expected as cartesian control offers a direct access to the foot position defined in cartesian space without relying on inverse kinematics. High-speed locomotion requires accurate footwork to maintain stability.
- Joint position tracking performance does not change significantly with the introduction of cartesian PD as is displayed in Fig. 9a and Fig. 9b. Fig. 9c confirms that pure Cartesian PD fails at accurate joint angle tracking. Unlike Joint PD, it relies on forward kinematics, leading to significant tracking deviations.
- High Cartesian gains increased stiffness, counterintuitively improving high-speed performance by reducing

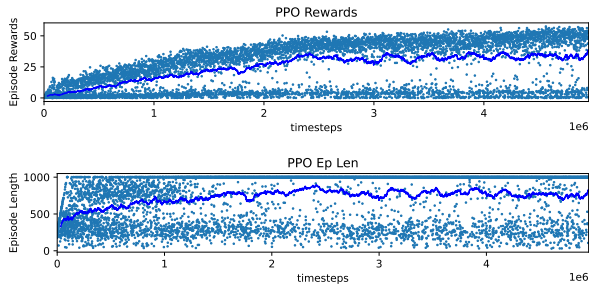


Fig. 5: Training metrics for the stairs task.

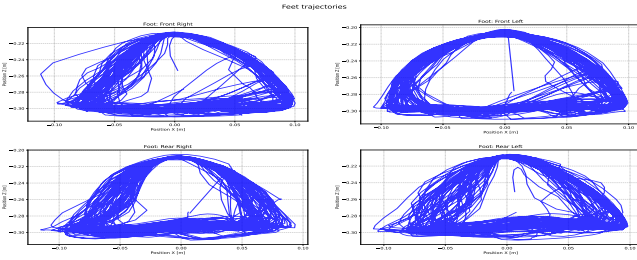


Fig. 6: Feet trajectories during stair climbing.

scuffing. However, excessive gains must be avoided, as they induce destabilizing foot position oscillations.

2) *Parameter Tuning for high and low velocity*: CPGs offer inherent stability for low-speed locomotion, but tuning them for high velocities is challenging. Detailed impacts of CPG hyperparameters on maximum velocity are discussed in Table V, with our optimal values listed in Table VI. Despite tuning, the system remains prone to instability, falling in approximately 20% of high-speed runs (mostly at takeoff). We found that adjusting the coupling strength from its default value of 1 further increased instability, leading us to abandon its optimization.

As shown in Table III, our controller exhibits excessively high CoT values (exceeding 2) compared to state-of-the-art CPG controllers, which achieve values below 1 [9]. This inefficiency stems from our exclusive focus on velocity during tuning, neglecting energy metrics—a limitation addressed later via RL. Specifically, high CoT is a byproduct of the high stiffness induced by $K_{d, cart}$ gains, which often forces motors to their torque limits.

The Bound gait proved most efficient during high-speed tests, although it achieved lower overall speeds due to the high stride duration required for stability. This duration prevents the energy-intensive “tiptoeing” seen in trotting or walking. However, at low speeds, the Bound gait exhibits a scuffing motion, dragging the hind feet and resulting in a very high CoT of 2.5.

Finally, while decreasing the duty ratio should theoretically enable faster “running” gaits, doing so in our experiments led to immediate instability. We attribute this to the lack of feedback mechanisms; without real-time sensory data to adjust limb timing and force distribution, the system cannot manage

the dynamic demands of low duty-ratio locomotion.

3) *Virtual Model Control*: VMC could significantly improve the CPG-performance in terms of direction following and robot stability.

First examining the xy -trajectory in Fig. 10a, note that in the run without VMC the robot immediately begins to deviate from the forward direction ($y = 0$). With VMC however, the robot does not experience such lateral displacements. Even in simulations where the VMC system began by deviating from the forward direction, it was quickly able to correct its heading and return to the intended path. Varying the rotational and translational reference values ψ_{des} and y_{des} , in theory would allow the robot to track any possible trajectory.

The body orientation evolution in Fig. 10b shows a clear improvement in roll stability, while pitch stability remains unchanged. Separate testing of the VMC components revealed that this improvement is primarily due to the lateral control, which intuitively aligns with its exclusive influence on hip joints and thus roll dynamics. Additionally, lateral VMC remains active throughout the entire stride, making it more influential than other components. In contrast, attitude control remained largely ineffective even with a tuning factor of $k_{att} = 300$. Further increasing this gain during high-speed tuning immediately caused instability and falls. This can be attributed to the system operating near the stability limit with highly stiff locomotion, where the asymmetric virtual forces introduced by attitude VMC destabilize the robot, whereas the symmetric inputs of lateral VMC remain stable.

4) *Potential Extensions and Future Improvements*: Several extensions could enhance the robustness of the CPG framework. First, a stumbling correction reflex could handle unexpected swing-leg collisions; detecting abnormal torques would trigger a corrective CPG signal to increase knee flexion, clearing obstacles as in [6].

Second, integrating force feedback—for instance, by adding ground reaction terms to the phase equation (Eq. 2) [10]—would allow dynamic modulation of stiffness and contact. This would improve load distribution and likely enable running gaits with duty ratios below 0.5.

Third, the framework could support dynamic gait transitions. Since CPGs allow gait changes via simple modifications of coupling parameters ϕ_{ij} , the robot could flexibly transition between walking and trotting.

Finally, implementing speed modulation by varying intrinsic frequencies ω_i would improve stability. Gradually increasing velocity from a stable, low-speed start would likely reduce the high failure rate currently observed at the beginning of runs.

B. RL Cartesian PD – Velocity

1) *Hyperparameter Sensitivity and Fine-tuning*: We observed high sensitivity to PPO hyperparameters. Contrary to standard practices, an entropy coefficient of 0.0 proved optimal, as non-zero values induced significant instability.

Regarding the learning rate, while a fixed $1 \cdot 10^{-4}$ was generally effective, it occasionally caused “catastrophic unlearning” near convergence (Fig. 15) due to overshooting. To rectify

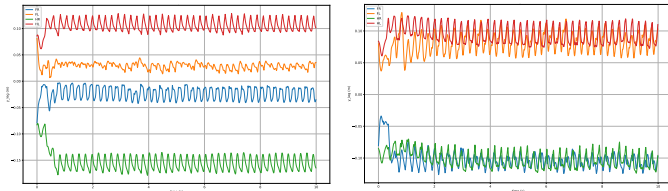
Parameter	Tuning Importance	Effect on Stability/Speed
ω_{swing}	Critical: Must balance with ω_{stance} .	High ω_{stance} or ω_{swing} can cause instability; generally low ω reduces overall speed.
ω_{stance}	Critical: Must balance with ω_{swing} .	Best to maintain a ratio of ω_{swing} and ω_{stance} of 2 – 4.
des_step_len	Critical: Can be scaled up with ω .	Shorter steps improve stability; longer steps increase speed but risk imbalance.
$ground_clearance$	High: No direct impact on speed, but critical to keep balance.	- High clearance induces excessive vertical motion $\rightarrow$ falling - Low clearance improves stability but causes scuffing if controller gains are too low (exact trajectory following important).
$ground_penetration$	High: Slightly helps with speed, but mainly to keep balance.	High penetration means robot kicks harder. This is beneficial for moving fast but quickly leads to falls $\rightarrow$ value best kept low
$kd, kdCartesian$	High: However, can lead to oscillations if pushed too far	Increases stiffness which at high speed reduces the chance of tripping $\rightarrow$ improves stability; Improves foot trajectory following (see Fig. 2) $\rightarrow$ avoid scuffing
α	Moderate: Affects only gait initiation (where most falls happen).	Low α eases into gait; high α risks imbalance but speeds up initiation. We kept α low to insure stable starts but sacrificing some speed.
$robot_height$	Moderate: impacts speed only indirectly, mostly for stability.	Low body height increases stability but also the CoT. <i>Trot/Walk:</i> deviations from default cause instant instability. <i>Bound:</i> Lower CoG necessary to balance.

TABLE V: Hyperparameters and their influence on speed and stability for CPG locomotion with trotting, bounding, and walking gaits.

these instances, we adopted a rollback strategy: we restore the last stable checkpoint saved prior to the collapse and concluded the training using a linear learning rate schedule decaying to zero.

2) *Controller Gains and Compliance:* We lowered the PD gains to $K_p^{cart} = 300, K_d^{cart} = 12$ compared to standard CPG experiments. This reduction compensates for the DRL policy’s inherent high-frequency noise (“jitter”). By reducing stiffness, we introduce mechanical compliance that acts as a low-pass filter, effectively dampening these oscillations to prevent rigid ground impacts and ensure stability.

3) *Curriculum Learning:* Reinforcement learning was useful in the velocity tracking controller since we are able to train it in stages and fix unwanted behaviors by adjusting the reward. In the first training, the policy mainly learns a stable locomotion and repeatable gait. In order to achieve a higher reference speed, we fine-tune it to adapt the duty ratio to achieve this desired speed. In the final stage, we add a stance-related penalty to significantly reduce the undesired width asymmetry while maintaining the same tracking objective. Overall, using a curriculum learning approach makes reference-velocity tracking easier because stability is learned first, and then it can be fine-tuned to achieve a cleaner gait behavior and follow a different, possibly higher velocity.



(a) Lateral leg positions before re-training (b) Lateral leg positions after re-training

Fig. 7: Leg stance comparison

Fig. 7 compares the foot placement between the second and third training stage. Before re-training the legs oscillate around different lateral offsets, indicating a biased, asymmetric stance width even though the gait is repeatable. After adding the stance penalty, the oscillations remain but are re-centered at similar mean lateral positions, reducing the width asymmetry while keeping the same velocity-tracking performance.

4) *Cartesian-PD:* We are also able to achieve this natural and robust gait coordination by using a Cartesian-PD instead of joint-space PD, because it directly lets the policy adjust the robot’s legs’ motion directly in the world frame instead of indirectly through joint angles.

5) *Cost of Transport:* The RL velocity controller achieves high speeds while maintaining energy efficiency, as its reward function penalizes excessive torques. Comparing results in Table III confirms that the RL policy outperforms manually tuned gaits. The comparison with the bounding gait is particularly striking: at an equivalent CoT of 1.2, the RL controller reaches higher velocities, despite the typical trend of CoT increasing with speed [7]. This efficiency is driven by a reduced duty ratio—a dynamic behavior the CPG-based controller could not achieve without force feedback—demonstrating the RL approach’s superior ability to balance speed and power consumption.

6) *Command at Test Time:* In this project, the policy was trained for a fixed target velocity. A straightforward extension to generalize this to any commanded velocity at test time, is by including the desired forward speed in the observation as in [7]. Then, during training, the command should be randomized across episodes or be updated periodically within an episode. This is important to allow a single policy to adapt its gait online based on the reference input.

C. RL CPG – Stairs

1) *Parameters:* For the CPG-based stair climbing we used the same and PPO hyperparameters as for the velocity training.

Later on we extended these by generalizing the rollback into a history-based fine-tuning technique for trained models (see IV-C9). Furthermore, we use the same cartesian PD gains and introduce the joint PD gains $K_p = 100$, $K_d = 2$.

2) *CPG-RL Architecture*: Standard end-to-end DRL often struggles to discover rhythmic gaits without complex reward shaping. To address this, we adopted a hierarchical CPG-RL architecture utilizing CPGs as rhythmic priors. This framework decouples gait generation from adaptation, allowing the RL agent to focus only on manipulating the oscillator parameters (μ, ω) rather than learning joint mechanics from scratch. This simplifies reward design. Since periodicity and foot clearance are intrinsic to CPG trajectories, we achieved robust climbing performance without the need for reward terms explicitly enforcing trajectory constraints.

3) *Sim-to-Real Strategy*: Transferring a policy learned in simulation to a real robot is challenging due to the *Sim-to-Real gap*, the discrepancy between the physics of the simulator (PyBullet) and the real world. To mitigate this, we adopted a Domain Randomization approach. By randomizing the physical and environmental parameters during training, the agent learns to be robust to variations, treating the real world as just another instance of the randomized simulation distributions. Based on our implementation, we introduced uncertainties in three key areas: robot dynamics, environment geometry, and sensor noise.

4) *Proprioceptive Adaptation to Stair Geometry*: Lacking exteroceptive perception, the agent utilizes proprioceptive feedback by leveraging the contacts with the stair risers at the end of the swing phase to infer the terrain profile, a behavior visible in Fig. 6. Interestingly, the policy tends to lock the foot deep into the step corner. While this approach may reduce energy efficiency compared to edge placement, it provides a robust localization method for blind locomotion. A comparison with biological strategies is shown in Fig. 16, highlighting how the robot uses physical interaction to overcome the absence of vision.

5) *Dynamics Randomization*: To account for modeling errors, we randomized the following dynamic properties at the start of each episode:

- **Base Mass & CoM**: A random payload $m_{add} \sim \mathcal{U}(0, 6)$ kg is attached at a variable offset (± 15 cm along x , ± 5 cm along y, z) to alter total inertia.
- **Ground Friction**: The lateral friction coefficient is sampled uniformly in $\mu \in [0.5, 1.0]$.
- **Motor Gains**: We injected multiplicative Gaussian noise ($\sigma \approx 2\%$) into all PD controller gains (K_p, K_d) to model actuator deviations.

It is worth noting that these disturbances were introduced only at the final stage of our pipeline. We hypothesize that training the robot with such randomizations from the very beginning could have yielded even more stable results.

6) *Environmental Randomization*: For the stairs traversal task, relying on perfect geometric knowledge leads to brittle policies. We randomized the dimensions of the stairs to force

the agent to learn a high-clearance gait capable of negotiating uneven terrain:

- **Step Height**: Randomized in $h_{step} \in [0.045, 0.052]$ m.
- **Step Width**: Randomized in $w_{step} \in [0.24, 0.26]$ m.

This variation prevents the robot from memorizing a fixed stride length and forces it to react to the terrain.

7) *Observation Noise*: To mimic sensor imperfections, we injected Gaussian noise into the observation vector $\mathbf{o}_t$, scaled to 1% of the observation limits. Crucially, for the history-based policy, this noise applies only to current sensor readings, leaving the action history deterministic.

8) *Generalization to Unseen Terrains*: Although trained exclusively on flat ground and randomized stairs, the policy exhibits remarkable zero-shot generalization to unseen environments, including ramps and rough terrains. This robustness stems from the aggressive Domain Randomization strategy, which induced a conservative, high-stepping gait capable of absorbing diverse surface irregularities. Qualitative results demonstrating these capabilities are provided in the supplementary videos `RL_SLOPES_0.71ms.mp4` and `RL_RANDOM_0.67ms.mp4`.

9) *Temporal Awareness and Future Work*: To enhance state estimation under partial observability, we augmented the observation space with a history buffer of the last 15 joint states $(q, \dot{q})$. This high-dimensional input is processed by a Multi-Layer Perceptron (MLP) featuring a funnel-shaped architecture with hidden layers of decreasing size (512, 256, 128).

Due to time constraints, training was limited to 4cm stairs using dynamics randomization. Consequently, the resulting policy exhibits a suboptimal, oscillating gait compared to the standard baseline. However, it demonstrates superior initial stability: unlike memory-less models which occasionally fail at startup, the history-aware agent consistently maintains balance during initialization. This suggests that with fine-tuning, this architecture holds the potential for greater robustness (see video `RL_STAIRS_H_0.57ms.mp4`).

V. CONCLUSION

This project highlighted the strengths and limitations of CPG-based and RL-trained locomotion. While CPGs ensure rhythmic stability and high speeds, achieving robust movement remains difficult. Our testing was restricted to high duty ratios, resulting in excessive CoTs. Although VMC improved directional and roll stability, pitch control, force feedback, and reflex strategies remain open challenges. RL models, using either Cartesian PD or CPG priors, successfully overcame these hurdles, enabling dynamic, efficient locomotion and robust stair climbing. Addressing the sim-to-real gap was central to our strategy; aggressive domain randomization ensured performance on unseen terrains. Future research should prioritize refining history buffer integration and exploring minimal observation spaces, such as those based solely on foot contacts [7].

REFERENCES

- [1] A. J. Ijspeert, A. Crespi, D. Ryczko, and J.-M. Cabelguen, "From swimming to walking with a salamander robot driven by a spinal cord model," *Science*, vol. 315, no. 5817, pp. 1416–1420, 2007.
- [2] L. Righetti and A. J. Ijspeert, "Pattern generators with sensory feedback for the control of quadruped locomotion," in *2008 IEEE International Conference on Robotics and Automation*, 2008, pp. 819–824.
- [3] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, "Learning agile and dynamic motor skills for legged robots," *Science Robotics*, vol. 4, no. 26, 2019.
- [4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [5] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International Conference on Machine Learning (PMLR)*, 2018, pp. 1861–1870.
- [6] M. Ajallooeian, S. Pouya, A. Sproewitz, and A. J. Ijspeert, "Central pattern generators augmented with virtual model control for quadruped rough terrain locomotion," in *2013 IEEE International Conference on Robotics and Automation*, 2013, pp. 3321–3328.
- [7] G. Bellegarda and A. Ijspeert, "Cpg-rl: Learning central pattern generators for quadruped locomotion," *IEEE Robotics and Automation Letters*, 2022.
- [8] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning quadrupedal locomotion over challenging terrain," *Science robotics*, vol. 5, no. 47, p. eabc5986, 2020.
- [9] G. Bellegarda, M. Shafiee, and A. Ijspeert, "Allgaits: Learning all quadruped gaits and transitions," 2024. [Online]. Available: <https://arxiv.org/abs/2411.04787>
- [10] D. Owaki, M. Goda, S. Miyazawa, and A. Ishiguro, "A minimal model describing hexapedal interlimb coordination: The tegotae-based approach," *Frontiers in Neurorobotics*, vol. 11, p. 29, 2017.
- [11] Gungho Dog Training. (2020) Dog is afraid of stairs (easy fix). YouTube. [Online]. Available: <https://www.youtube.com/watch?v=6MrOiHi04zc>

VI. USE OF GENERATIVE AI

Generative AI tools were employed primarily for the grammatical and stylistic refinement of the report, for generating boilerplate code related to data visualization, and as a coding assistant for implementing specific architectural features decided by the authors. AI tools were **not** used to design the control strategies or substitute the fundamental understanding of the course material. In general, AI served to accelerate secondary tasks and assist with the syntax of complex implementation details.

1) Most Useful Aspects

Report Refinement: Large Language Models (LLMs) like ChatGPT were instrumental in translating informal descriptions of our design choices into formal academic English. This was particularly useful for the *Discussion* and *Methodology* sections, where we needed to articulate the reasoning behind specific decisions or to summarize complex results within the page limit.

Example prompt: "Refine this paragraph to explain that we introduced a linear learning rate schedule because the fixed rate caused instability at convergence. Keep it academic and concise."

Result: A well-structured explanation that accurately conveyed our findings regarding the "unlearning" phenomenon.

Code Assistance & Plotting: Tools like GitHub Copilot significantly sped up the generation of plotting scripts, allowing us to focus on data analysis rather than Matplotlib syntax.

Example prompt: (Comment in code) `# Plot reward components`

Result: Immediate generation of the correct plotting function.

Implementation Support for Custom Architectures:

While the decision to augment the observation space with a history buffer was derived from personal study, AI tools were used to integrate this feature into our existing codebase. We provided the specific Python method responsible for computing observations and requested the necessary logic to manage a sliding window of past states.

Example prompt: "This is the function I want to modify, how do I implement and manage the history?"

Result: The AI analyzed the provided code snippet and generated the exact implementation for a deque-based buffer, correctly handling the concatenation of current and past observations.

Implementation Support for Interpretable Reward Shaping:

During training we observed an emerging stance-width asymmetry (front legs consistently closer together than hind legs), which we identified as an undesirable behavior for a stable and transferable gait. We therefore decided to add an explicit stance regularization penalty (nominal lateral foot placement, front–hind width consistency, and left–right symmetry). AI tools were then used to help translate this intent into a correct implementation using the existing kinematics interfaces (e.g., extracting foot positions in the leg frame and forming the penalty terms).

Example prompt: "We observe front–hind stance width asymmetry; add a penalty that keeps each foot close to nominal y in the leg frame, penalizes front–hind width mismatch, and encourages left–right symmetry. Use `ComputeJacobianAndPosition()` to get foot positions."

Result: A compact stance regularization term integrated into the reward function

2) Least Useful Aspects

Generative AI was less effective when dealing with the specific physics tuning of the simulation environment. Prompts asking for specific PPO hyperparameter values for our custom robot model often yielded generic values that resulted in poor performance. The fine-tuning process required manual empirical testing.

3) **Impact on Project Development**

The use of AI tools streamlined the development process, particularly in the post-processing of data and the drafting of the final report. This efficiency allowed us to dedicate more time to the task designing the reward function and conducting training runs to achieve robust locomotion.

4) **Impact on Learning**

AI tools did not hinder our learning process but acted more as an interactive tutor. When encountering complex aspects of the PPO algorithm (such as the role of the value function loss or entropy regularization), we used LLMs to obtain simplified explanations and analogies. This reinforced our theoretical understanding, allowing us to make more informed decisions during the hyperparameter tuning phase.

VII. APPENDIX

A. CPG

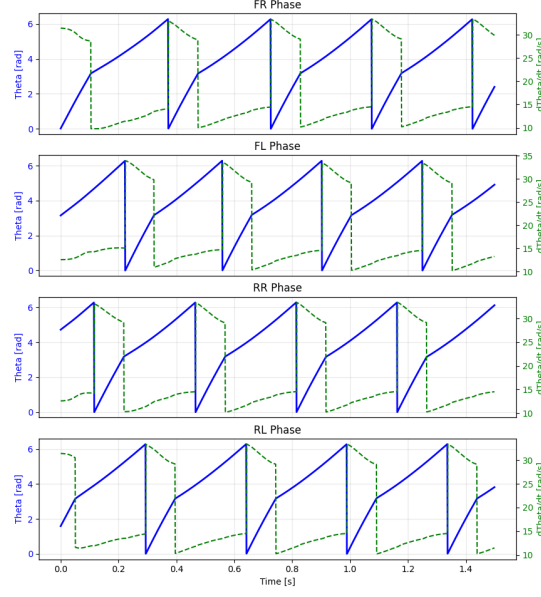


Fig. 8: Development of the CPG states for a walking gait

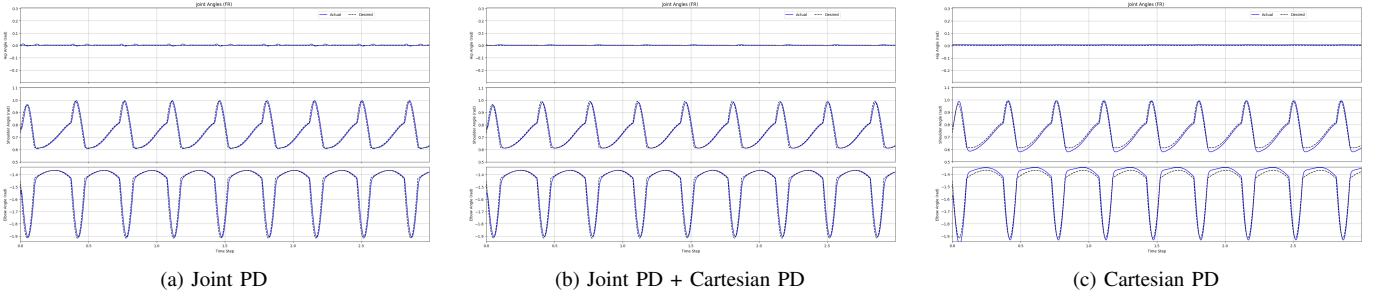
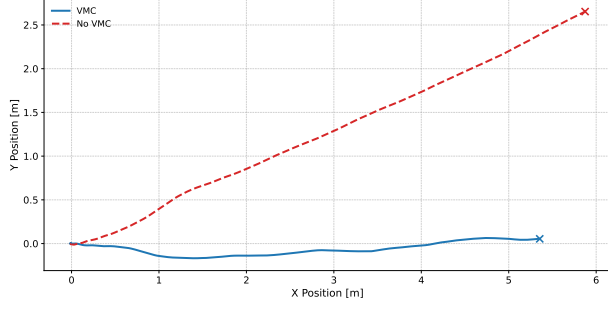


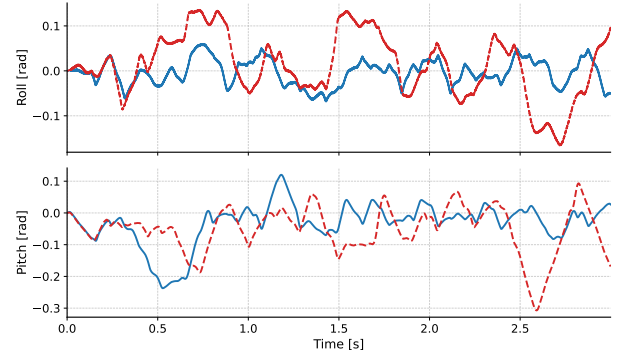
Fig. 9: Joint angle evolution for different combinations of PD control

Parameter	Trot		Walk		Bound	
	High	Low	High	Low	High	Low
omega_swing	$14.85 \cdot 2\pi$	$2.4 \cdot 2\pi$	$12 \cdot 2\pi$	$4.7 \cdot 2\pi$	$3.6 \cdot 2\pi$	
omega_stance	$5.4 \cdot 2\pi$	$1.2 \cdot 2\pi$	$6 \cdot 2\pi$	$1.7 \cdot 2\pi$	$1.8 \cdot 2\pi$	$0.8 \cdot 2\pi$
des_step_len	0.095	0.03	0.12	0.03	0.13	0.09
ground_clearance	0.01	0.02	0.01	0.03	0.04	0.03
ground_penetration	0.01					
kd, kdCartesian	700 and 70					
alpha	5	10	10	5	15	
robot_height	0.28	0.30	0.30		0.23	

TABLE VI: Hopf oscillator parameter values for trotting, walking, and bounding gaits.



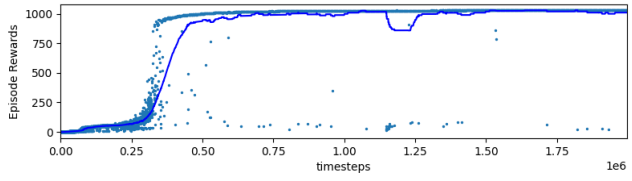
(a) xy -trajectory



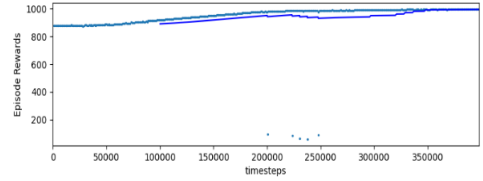
(b) Roll and pitch body orientation comparisons

Fig. 10: Trajectory comparison for a 3-second walk simulation with and without VMC

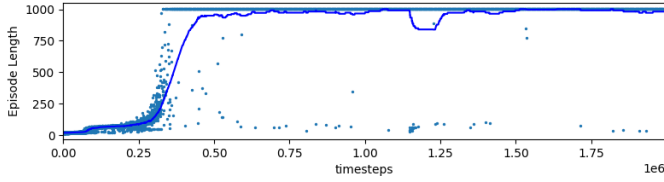
B. RL-Velocity



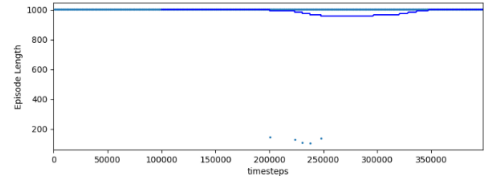
(a) Train 1: Reward



(b) Train 3: Reward

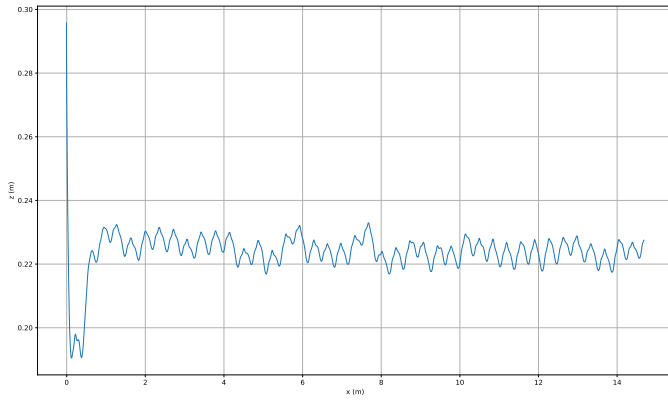


(c) Train 1: Ep. length

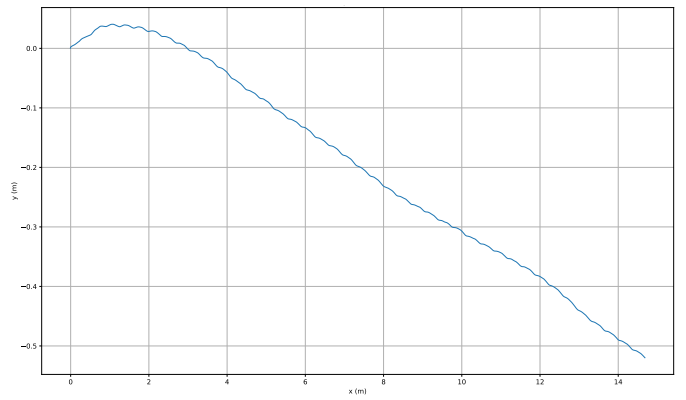


(d) Train 3: Ep. length

Fig. 11: PPO learning curves for velocity controller. Training 1: Baseline, Training 3: Stance correction



(a) Side view (XZ)



(b) Top-down view (XY)

Fig. 12: Robot trajectory during RL-trained forward locomotion

C. RL-Stairs

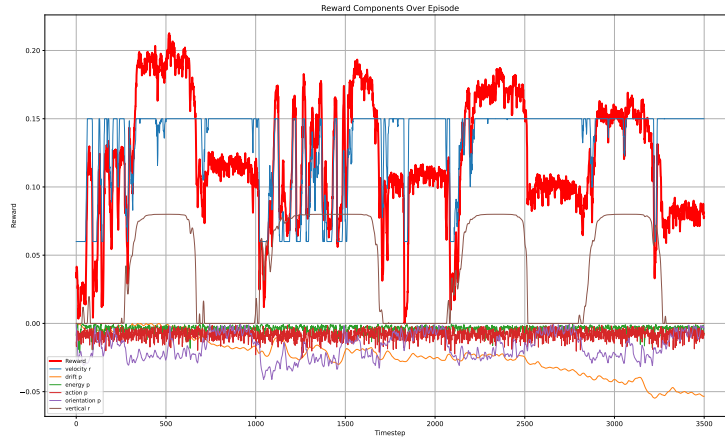
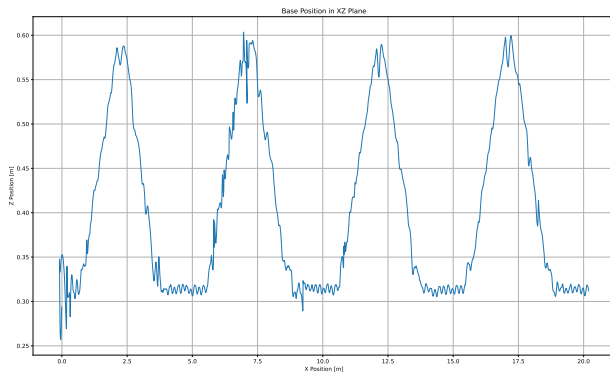
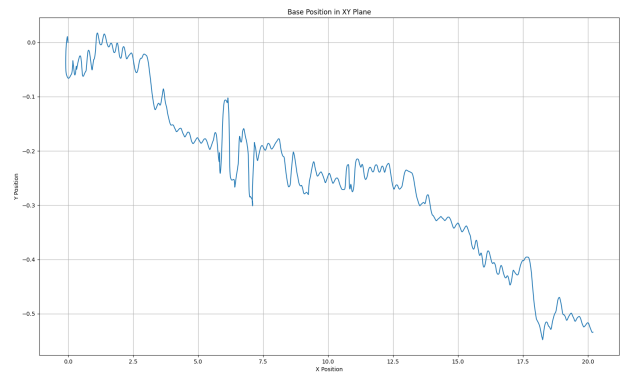


Fig. 13: Decomposition of the reward function terms over an episode.



(a) Side view (XZ plane)



(b) Top-down view (XY plane)

Fig. 14: Trajectory of the robot's base during stair climbing.

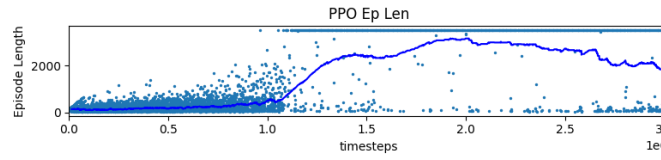
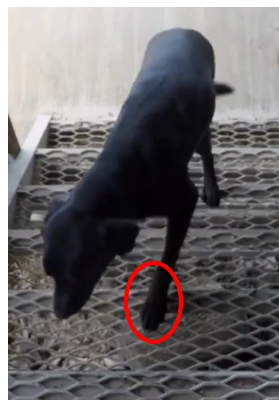


Fig. 15: Example of episode mean length collapse (unlearning) occurring at the end of training with a fixed learning rate.



(a) Biological dog foot placement.



(b) Robot foot placement in corner.

Fig. 16: Comparison of biological [11] vs. robotic foot placement strategy on stairs. Red circles highlight the contact point relative to the stair riser, showing how the robot exploits physical contact for localization in the blind setting.